Strecht Simulados

Resumo de bolso DVA-C02

Domínios cobrados, as dicas de prova mais importantes e 10 questões comentadas para treinar.

390	6	72%
questões no curso	provas completas	nota de corte

Como é a prova DVA-C02

Desenvolvimento com os serviços da AWS

Segurança

Implantação

Solução de problemas e otimização

Material independente de preparação, sem afiliação ou endosso de Amazon Web Services (AWS). As questões são inéditas e não reproduzem a prova real.

Pronto para o exame DVA-C02?

Treine com as 390 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →

Dicas de prova por domínio

Desenvolvimento com os serviços da AWS

- Para workloads com picos imprevisíveis e necessidade de retenção de mensagens, prefira SNS + SQS + Lambda em vez de SNS chamando endpoints HTTP diretamente. Use SQS padrão para alta taxa de throughput, backoff e reentregas, e configure DLQ sempre que possível para facilitar troubleshooting de falhas de processamento.
- Para dados de sessão, metadados de usuário, carrinho de compras e outros dados chave-valor com alta taxa de leitura/escrita e baixa latência, priorize serviços totalmente gerenciados como o DynamoDB. Pense em RDS para dados relacionais e em S3 para armazenamento de objetos e arquivos.
- Para consumir SQS de forma eficiente em aplicações escaláveis, combine: long polling (WaitTimeSeconds > 0), `MaxNumberOfMessages` alto (até 10) e ajuste do `VisibilityTimeout` de acordo com o tempo de processamento da sua aplicação.
- Quando o enunciado mencionar processamento assíncrono, escalabilidade automática e ausência de gerenciamento de servidores, pense em combinar serviços serverless e totalmente gerenciados, como API Gateway, AWS Lambda, Amazon SQS ou Amazon EventBridge.
- Sempre que um serviço da AWS (como Lambda, ECS, EC2) precisar acessar outros serviços, pense primeiro em IAM Roles com políticas de menor privilégio, de preferência definidas como código (CloudFormation, CDK, Terraform). Evite credenciais estáticas e permissões amplas (como '*:*' ou 'FullAccess') em produção.
- Sempre que um serviço gerenciado da AWS (como Lambda, ECS, EC2) precisar acessar outro serviço da AWS, prefira usar IAM Roles e políticas de menor privilégio em vez de credenciais estáticas (access key/secret key). Isso simplifica o gerenciamento de segurança e reduz o risco de vazamento de credenciais.
- Para workloads em VPC privada que precisam acessar S3 ou DynamoDB sem sair para a internet, pense primeiro em endpoints de VPC (gateway ou interface). Para aplicações em ECS ou Lambda, prefira sempre IAM roles (task role ou execution role) em vez de usuários IAM com chaves estáticas.
- Quando precisar processar arquivos novos em um bucket S3 de forma automática e serverless, pense primeiro em notificações de evento do S3 disparando funções Lambda diretamente. Apenas adicione SQS, SNS ou outros componentes se houver requisitos específicos (reprocessamento, fan-out, buffering, etc.).

Troubleshooting e Otimização

- Respostas vazias de ReceiveMessage = ligar long polling (wait time até 20 s).
- Sempre que o CloudWatch indicar aumento em 'Init Duration' com 'Duration' estável, pense em cold starts. Em ambientes de produção sensíveis à latência, use provisioned concurrency em conjunto com aliases estáveis e, se necessário, ajuste memória apenas após analisar métricas detalhadas de performance.

- Em Lambda, sempre diferencie: (1) latência por cold start, (2) latência por CPU insuficiente e (3) latência por IO externo. Use provisioned concurrency para reduzir cold start, ajuste memória/CPU com base em métricas reais e otimize acesso a serviços externos (reutilização de conexões, batch, paralelismo).
- Use variáveis de ambiente para controlar o nível de log (por exemplo, 'LOG_LEVEL=info|debug|error') e condicionar logs verbosos somente quando realmente necessário. Em produção, mantenha logs mais enxutos para reduzir latência e custos de CloudWatch Logs.
- Sempre que encontrar erros transitórios em serviços AWS (como throttling em DynamoDB, erros 5xx temporários em APIs, etc.), pense primeiro em implementar backoff exponencial com jitter antes de escalar recursos. Essa abordagem é barata, segue boas práticas e costuma estabilizar o sistema durante picos de carga.
- Ao usar Kinesis como gatilho de Lambda, monitore métricas como IteratorAge, Duration e Throttles. Ajuste o batch size e a concorrência para balancear vazão e latência. Use Partial Batch Response sempre que possível para evitar o reprocessamento de registros bem-sucedidos quando apenas alguns itens do lote falham.
- Ao integrar SQS, Lambda e DynamoDB, pense sempre em controle de taxa (limitar concorrência e tamanho de lote), estratégias de retry com backoff exponencial e uso de DLQ para tratar mensagens problemáticas. Otimizar não é só 'aumentar capacidade'; muitas vezes é controlar melhor o fluxo de entrada.
- Ao ver questões envolvendo Lambda + RDS com problemas de latência e limites de conexão, pense em dois padrões: Provisioned Concurrency para reduzir cold start e RDS Proxy para gerenciar e reutilizar conexões. Dimensionar apenas o banco normalmente não é a resposta mais eficiente em custo.

Segurança

- Sempre que possível, substitua 'Resource': '*' pelo ARN exato do recurso (ou por um conjunto mínimo de ARNs) ao conceder permissões a funções Lambda, roles de serviço ou usuários. Para DynamoDB, use 'arn:aws:dynamodb:::table/' para políticas mais seguras e alinhadas ao princípio de menor privilégio.
- Quando a questão mencionar rotação automática de segredos, auditoria detalhada e evitar exposição em pipelines, pense primeiro em AWS Secrets Manager. Quando o foco for apenas configuração segura simples (sem rotação complexa), o Parameter Store pode ser suficiente.
- Quando o enunciado enfatizar rotação automática de segredos, integração simples com aplicações e auditoria de acessos, pense primeiro em AWS Secrets Manager. Já o Parameter Store é excelente para configurações e segredos menos dinâmicos, ou quando custo menor for um fator decisivo.
- Para integrações com Cognito em APIs serverless, priorize o uso de JWT Authorizers (Cognito Authorizer) no API Gateway. Isso garante validação padronizada de tokens, reduz código de segurança dentro da Lambda e facilita implementação de RBAC usando claims (grupos, escopos) e políticas baseadas em atributos.

- Para chamadas internas entre serviços na mesma conta (EC2, Lambda, ECS) prefira sempre o uso de roles IAM e credenciais temporárias fornecidas pelo AWS STS. Evite chaves estáticas (Access Keys) sempre que possível e use o modelo de permissões mínimo necessário (least privilege) nas políticas associadas ao role.
- Sempre que for conceder acesso a segredos (Parameter Store ou Secrets Manager), pense em: (1) princípio de menor privilégio; (2) anexar permissões ao papel de execução (role) do recurso que consome o segredo; (3) restringir o acesso por ARN do parâmetro/segredo, nunca por wildcard desnecessário.
- Para workloads em contêiner no EKS, lembre-se: 'Pod não usa chave, usa função por service account'. Quando a questão mencionar EKS + acesso a serviços AWS + evitar chaves estáticas, pense imediatamente em IRSA (IAM Roles for Service Accounts) e princípio de menor privilégio em nível de pod/service account.
- Para workloads em containers (EKS) que precisam de acesso autenticado à AWS sem chaves estáticas, pense em IRSA. Para EC2, lembre-se de Instance Profiles; para Lambda, use a execution role. Sempre busque roles associadas ao runtime em vez de armazenar chaves em código ou secrets.

Implantação

- Para cenários de ECS + Fargate em que você precisa de controle refinado de tráfego, health checks e rollback automático, pense em: ECS Service + Application Load Balancer + CodeDeploy (deployment type blue/green). Esse trio costuma aparecer em questões envolvendo zero-downtime e rollback automático.
- Para lembrar: em Elastic Beanstalk, blue/green = dois ambientes distintos + swap de CNAME. Métodos 'rolling' e 'all at once' são estratégias de atualização dentro de um mesmo ambiente, não substituem um verdadeiro blue/green deployment.
- Para aplicações serverless, prefira AWS SAM ou CloudFormation em vez de scripts manuais. Templates permitem replicar o mesmo padrão em várias contas/regiões com um único comando (por exemplo, 'sam deploy' ou 'cloudformation deploy'), além de facilitar rollback e controle de versão.
- Para aplicações em ECS com requisitos de alta disponibilidade, combine CodePipeline + CodeDeploy + ECS usando Blue/Green. Sempre configure health checks adequados no ALB/NLB e revise limites de recursos (CPU/Memória), capacidade do cluster e timeouts do deployment antes de ajustar parâmetros agressivos de rollout.
- Para múltiplos ambientes (dev, qa, prod) com CloudFormation, prefira um único template parametrizado + múltiplas stacks. Use parâmetros, Mappings, Conditionals e, quando necessário, SSM Parameter Store ou Secrets Manager para separar valores sensíveis e específicos de ambiente.
- Para implantações canário ou blue/green em ECS com rollback automático, pense em AWS CodeDeploy como o componente central de implantação dentro de um pipeline CodePipeline. CodeBuild cuida do build e push da imagem (por exemplo, para ECR), enquanto CodeDeploy aplica a estratégia de implantação no ECS.
- Para implantações controladas com validações antes de expor usuários à nova versão em ECS, pense em CodeDeploy blue/green + ALB. Sempre que o enunciado falar em testes de pós-implantação,

hooks e rollback automático com mínimo esforço operacional, CodeDeploy é um forte candidato.

- Para implantações quase sem downtime em ECS, pense em blue/green com CodeDeploy quando o requisito envolver controle fino de tráfego e rollback simplificado. Lembre-se: rolling update do ECS é mais simples, mas menos sofisticado para cenários que exigem duas versões ativas e roteamento controlado.

10 questões para treinar

Responda antes de olhar o gabarito, no fim do material.

Questão 1 · Desenvolvimento com os serviços da AWS

Uma empresa está desenvolvendo um backend serverless em AWS Lambda para processar requisições HTTP recebidas via Amazon API Gateway. Em produção, o tempo de resposta da API está variando bastante, principalmente nas primeiras requisições após períodos de inatividade. O desenvolvedor quer reduzir essa variabilidade e melhorar a previsibilidade da latência, mantendo o modelo serverless. Qual solução é MAIS adequada para esse cenário?

A. Usar Provisioned concurrency na função crítica e ajuste de memória para reduzir o tempo de execução
B. Migrar o backend para uma aplicação em instâncias Amazon EC2 por trás de um Application Load Balancer para eliminar cold starts
C. Aumentar apenas o timeout da função Lambda e configurar o API Gateway para usar cache em todas as rotas
D. Habilitar DLQ (Dead Letter Queue) para a função Lambda e diminuir o timeout

Questão 2 · Desenvolvimento com os serviços da AWS

Um desenvolvedor está criando uma aplicação que armazena e processa objetos no Amazon S3. A aplicação deverá executar diferentes ações dependendo do tipo de evento gerado no bucket (por exemplo, criação de objeto com um prefixo específico, exclusão lógica via versão, ou restauração de objeto arquivado). O desenvolvedor quer minimizar o código de roteamento dentro da lógica da aplicação e deseja que o próprio serviço de mensageria faça o fan-out adequado para diferentes consumidores. Qual abordagem é a mais adequada para esse cenário?

A. Configurar o bucket S3 para enviar eventos diretamente para uma única fila Amazon SQS e implementar toda a lógica de roteamento dentro da aplicação consumidora.
B. Habilitar o Amazon EventBridge para ler diretamente os objetos do S3 e encaminhá-los automaticamente para diferentes APIs HTTP, sem necessidade de configuração adicional.
C. Usar eventos do S3 publicados num tópico SNS que distribui às filas SQS de cada componente com filtros de assinatura
D. Configurar o bucket S3 para invocar diretamente uma função AWS Lambda única, que então envia mensagens.

Questão 3 · Implantação

Um desenvolvedor está configurando um pipeline de implantação no AWS CodePipeline para uma aplicação web em contêineres executada no Amazon ECS com Fargate. O código-fonte está no AWS CodeCommit, os testes automatizados rodam no AWS CodeBuild e a imagem é armazenada no Amazon ECR. O desenvolvedor quer garantir que, em caso de falha na nova versão, o tráfego volte automaticamente para a versão anterior, minimizando downtime e risco de erro humano. Qual é a abordagem MAIS adequada para atender a esse requisito usando serviços gerenciados da AWS?

A. Configurar o AWS CodePipeline com estágios de Source (CodeCommit), Build (CodeBuild) e Deploy usando o AWS CodeDeploy com tipo de implantação Blue/Green para ECS, habilitando validações pós-implantação e rollback automático baseado em falhas em alarmes do Amazon CloudWatch.
B. Configurar o AWS CodePipeline com estágios de Source (CodeCommit), Build (CodeBuild) e Deploy (ECS) usando implantação do tipo Rolling Update, e fazer rollback manual caso a nova versão apresente falhas.
C. Configurar o AWS CodePipeline com estágios de Source (CodeCommit), Build (CodeBuild) e Deploy usando AWS CloudFormation para atualizar o serviço ECS com política de implantação All-at-once, e usar Change Sets para reverter à mão em caso de falhas.
D. Configurar o AWS CodePipeline com estágios de Source (CodeCommit) e Build (CodeBuild) só, e agendar um script em uma instância Amazon EC2 via cron para fazer o deploy no ECS com a AWS CLI e reverter à mão em caso de falhas.

Questão 4 · Desenvolvimento com os serviços da AWS

Um time de desenvolvimento está construindo uma aplicação serverless que receberá eventos de diversos produtores (IoT, serviços internos e aplicações de terceiros) por meio do Amazon EventBridge. Os desenvolvedores precisam testar em ambiente de desenvolvimento novas regras de roteamento e transformações de eventos sem impactar a produção, mas também desejam reutilizar o máximo possível da infraestrutura já criada, incluindo esquemas, barramentos e regras. Qual é a abordagem MAIS apropriada para atender a esses requisitos?

A. Criar um novo event bus personalizado em produção, apontar temporariamente os produtores para esse novo bus para testes e, após validar, mover as regras para o event bus original
B. Utilizar AWS SAM ou AWS CDK para definir a infraestrutura como código e criar stacks separados para desenvolvimento e produção, cada um com seu próprio event bus, regras, permissões e targets
C. Habilitar o recurso de archive e replay do EventBridge no event bus de produção e usar o replay para testar novas regras diretamente na conta de produção
D. Criar regras condicionais no event bus de produção que enviem eventos de teste para targets de desenvolvimento e eventos reais para targets de produção, diferenciando-os só por um campo de payload

Questão 5 · Implantação

Um time de desenvolvimento deseja automatizar a implantação de uma aplicação em contêiner no Amazon ECS (usando Fargate) sempre que um novo commit é feito no branch 'main' de um repositório GitHub. A equipe quer evitar gerenciar servidores de build e prefere usar serviços totalmente gerenciados da AWS. Qual abordagem atende a esses requisitos com o menor esforço operacional?

A. Criar instâncias EC2 dedicadas para build, configurar webhooks do GitHub para disparar scripts de shell que executam docker build, docker push para o Amazon ECR e chamar à mão a atualização do serviço ECS
B. Configurar um pipeline no AWS CodePipeline integrado ao GitHub como fonte, usar AWS CodeBuild para build da imagem Docker e enviar para o Amazon ECR, seguido de uma ação de implantação no Amazon ECS
C. Usar o AWS CodeCommit como repositório de código-fonte, migrando o código do GitHub, e configurar um pipeline com CodePipeline e CodeDeploy para implantação em ECS
D. Criar um job do AWS Batch que é disparado por eventos do Amazon EventBridge ao detectar novos commits no GitHub, para construir a imagem e atualizar o serviço ECS

Questão 6 · Desenvolvimento com os serviços da AWS

Um desenvolvedor está criando um backend sem servidor que processa eventos do Amazon S3 usando AWS Lambda. Em ambiente de testes, a função funciona corretamente, mas em produção, sob alta carga, alguns eventos do S3 não são processados e outros são processados mais de uma vez. O desenvolvedor precisa adaptar o código para lidar corretamente com esse comportamento e garantir consistência no processamento. O que ele deve fazer?

A. Tornar a função idempotente, tolerando eventos repetidos, atrasados ou fora de ordem
B. Ativar VPC na função Lambda, garantindo maior estabilidade de rede e, assim, eliminando a possibilidade de perda ou duplicação de eventos.
C. Configurar a função Lambda para o modo síncrono, garantindo que cada evento seja processado exatamente uma vez antes de retornar sucesso.
D. Aumentar o tempo máximo de execução (timeout) da função Lambda para evitar que eventos sejam descartados durante picos de carga.

Questão 7 · Segurança

Uma aplicação de processamento de pedidos em execução no Amazon ECS (Fargate) precisa ler e gravar dados em uma tabela do Amazon DynamoDB. Os requisitos de segurança da empresa determinam que nenhuma chave de acesso da AWS seja armazenada no código, em variáveis de ambiente ou em arquivos de configuração. Além disso, a solução deve seguir o princípio do menor privilégio e ser simples de gerenciar. Como o desenvolvedor deve configurar o acesso da aplicação ao DynamoDB?

A. Anexar uma política gerenciada 'AmazonDynamoDBFullAccess' diretamente ao usuário raiz da conta e configurar o ECS para assumir automaticamente essas permissões.
B. Criar um usuário do IAM com política de acesso ao DynamoDB, gerar chaves de acesso e armazená-las criptografadas no AWS Systems Manager Parameter Store para a aplicação ler em tempo de execução.
C. Criar uma função do IAM (IAM Role) para a tarefa do ECS com uma política que permita somente as ações necessárias no DynamoDB e associar essa função à definição da tarefa do Fargate.
D. Criar uma função do IAM para o serviço Amazon ECS, anexar uma política com permissões amplas ao DynamoDB e compartilhar essa função entre todas as aplicações em ECS para simplificar o gerenciamento.

Questão 8 · Desenvolvimento com os serviços da AWS

Um desenvolvedor precisa processar arquivos de imagem enviados por usuários para um bucket Amazon S3. Sempre que um novo objeto JPG for criado em um prefixo específico, o código deve ser executado para gerar miniaturas e salvar o resultado em outro bucket. O desenvolvedor quer minimizar a administração de servidores, garantir escalabilidade automática e manter o custo baixo. Qual é a abordagem MAIS adequada?

A. Configurar uma função AWS Lambda acionada por eventos de criação de objeto no bucket S3, processar a imagem e salvar a miniatura em outro bucket S3.
B. Utilizar um AWS Step Functions State Machine que é executado manualmente pelo desenvolvedor sempre que novas imagens forem enviadas ao bucket S3.
C. Implantar um contêiner no Amazon ECS com uma tarefa agendada para listar objetos no bucket S3 e processar as imagens encontradas.
D. Criar uma instância Amazon EC2 com um script em loop verificando o bucket S3 periodicamente usando cron para detectar novos arquivos e processá-los.

Questão 9 · Segurança

Uma empresa precisa armazenar segredos de aplicação (como chaves de API e strings de conexão) usados por funções AWS Lambda. Os segredos devem ser criptografados em repouso com chaves gerenciadas pela AWS e a aplicação deve conseguir recuperá-los em tempo de execução com baixa latência e controle de acesso detalhado via IAM. Qual serviço da AWS atende melhor a esses requisitos?

A. AWS Secrets Manager
B. AWS KMS
C. AWS Systems Manager Parameter Store (parâmetros seguros)
D. Amazon S3 com criptografia do lado do servidor (SSE-S3)

Questão 10 · Desenvolvimento com os serviços da AWS

Um desenvolvedor está construindo uma aplicação serverless que consome mensagens de uma fila do Amazon SQS e grava dados em uma tabela do Amazon DynamoDB. A aplicação usa uma função AWS Lambda escrita em Python. O desenvolvedor deseja reduzir o tempo de inicialização da função e melhorar a performance sob alta carga, mantendo os custos sob controle. Qual abordagem atende melhor a esses requisitos?

A. Configurar a função Lambda com memória mínima e sem variáveis de ambiente, para forçar a AWS a reutilizar o mesmo ambiente de execução por mais tempo.
B. Aumentar o timeout da função Lambda e desligar a concorrência reservada, permitindo que a AWS escale automaticamente sem limites.
C. Ativar Lambda provisioned concurrency e usar AWS SDK v3 modular (boto3 com importações específicas) junto com conexões reutilizáveis para o DynamoDB fora do handler.
D. Configurar a função Lambda para ser invocada só de dentro de uma VPC com NAT Gateway, garantindo que o cold start seja sempre reduzido.

Gabarito comentado

1. A) Usar Provisioned concurrency na função crítica e ajuste de memória para reduzir o tempo de execução
Em provas da AWS, quando o problema é latência imprevisível em Lambda (especialmente após períodos de inatividade), pense em "provisioned concurrency". Se a exigência é previsibilidade de tempo de resposta (picos conhecidos, low-latency, endpoints críticos), a provisão de concorrência é o mecanismo padrão. Ajuste de memória também é uma alavanca importante de performance em Lambda.

2. C) Usar eventos do S3 publicados num tópico SNS que distribui às filas SQS de cada componente com filtros de assinatura
Quando precisar distribuir eventos para múltiplos consumidores com diferentes necessidades de filtragem, pense no padrão S3 -> SNS -> SQS/Lambda. O SNS faz o fan-out e o filtro por mensagem, enquanto SQS ou Lambda ficam responsáveis pelo processamento desacoplado em cada fluxo.

3. A) Configurar o AWS CodePipeline com estágios de Source (CodeCommit), Build (CodeBuild) e Deploy usando o AWS CodeDeploy com tipo de implantação Blue/Green para ECS, habilitando validações pós-implantação e rollback automático baseado em falhas em alarmes do Amazon CloudWatch.
Para ECS e EKS, quando o requisito inclui rollback automático e controle refinado de tráfego entre versões, pense em integrações do AWS CodeDeploy com estratégias Blue/Green ou Canary. Lembre também de associar alarmes do Amazon CloudWatch ao CodeDeploy para que falhas de saúde da aplicação disparem o rollback automaticamente.

4. B) Utilizar AWS SAM ou AWS CDK para definir a infraestrutura como código e criar stacks separados para desenvolvimento e produção, cada um com seu próprio event bus, regras, permissões e targets
Sempre que possível, modele sua arquitetura serverless com infraestrutura como código e crie ambientes separados (dev, test, prod) em contas distintas ou, no mínimo, em stacks separados. Reutilize o código dos templates, não os recursos em si, entre ambientes. Isso reduz riscos e facilita automação com CI/CD.

5. B) Configurar um pipeline no AWS CodePipeline integrado ao GitHub como fonte, usar AWS CodeBuild para build da imagem Docker e enviar para o Amazon ECR, seguido de uma ação de implantação no Amazon ECS
Para cenários de CI/CD em que o código está no GitHub e o alvo é ECS ou Lambda, pense em: CodePipeline (orquestração) + CodeBuild (build/teste) + ECR (imagens) + ECS/Lambda (deploy). Isso tende a ser a combinação mais cobrada no exame para fluxos totalmente gerenciados.

6. A) Tornar a função idempotente, tolerando eventos repetidos, atrasados ou fora de ordem
Sempre que usar integrações de eventos assíncronos na AWS (como S3, SNS, EventBridge ou invocações assíncronas de Lambda), assuma o modelo de entrega at-least-once e projete suas funções para serem idempotentes. Use chaves de deduplicação (IDs de evento, hashes de payload, chaves de negócio) e verifique se o processamento já ocorreu antes de aplicar efeitos permanentes (gravações em banco, chamada

7. C) Criar uma função do IAM (IAM Role) para a tarefa do ECS com uma política que permita somente as ações necessárias no DynamoDB e associar essa função à definição da tarefa do Fargate.
Sempre que um serviço da AWS (como ECS, Lambda ou EC2) precisar acessar outros serviços, pense primeiro em funções do IAM (roles) com políticas mínimas necessárias, em vez de criar usuários do IAM com chaves de acesso estáticas.

8. A) Configurar uma função AWS Lambda acionada por eventos de criação de objeto no bucket S3, processar a imagem e salvar a miniatura em outro bucket S3.
Quando o requisito envolver 'acionar código sempre que um objeto for criado no S3', pense primeiro em eventos

do S3 integrados ao AWS Lambda. Essa combinação é padrão de arquitetura serverless para processamento de arquivos, reduzindo custo e esforço operacional.

9. A) AWS Secrets Manager
Quando a questão mencionar explicitamente 'segredos de aplicação', 'rotação de segredos', 'integração com Lambda/RDS' e controle de acesso via IAM, pense primeiro em AWS Secrets Manager. Se a ênfase estiver em parâmetros de configuração (não necessariamente segredos) e custo menor, considere SSM Parameter Store.

10. C) Ativar Lambda provisioned concurrency e usar AWS SDK v3 modular (boto3 com importações específicas) junto com conexões reutilizáveis para o DynamoDB fora do handler.
Para reduzir cold starts em Lambda: use provisioned concurrency para endpoints de baixa latência, inicialize clientes de SDK fora do handler, importe apenas o que for necessário no código, e evite colocar a função em VPC se não precisar acessar recursos privados.

Pronto para o exame DVA-C02?

Treine com as 390 questões em 6 provas completas: tempo do exame, explicação de cada alternativa e link da documentação para revisar.

Ver as 6 provas completas e o cupom do mês →